necessary services for the SQLite provider and provides the connection to the in-memory database. Because you're passing this options object in to an instance of `AppDbContext`, all calls to the `DbContext` result in calls to the in-memory database provider.

Listing 36.13 Using the in-memory database provider to test an EF Core `DbContext`

```
[Fact]
public void GetRecipeDetails_CanLoadFromContext()
{
    var connection = new SqliteConnection("DataSource=:memory:");    ❶
    connection.Open();                                               ❷

    var options = new DbContextOptionsBuilder<AppDbContext>()        ❸
        .UseSqlite(connection)                                       ❸
        .Options;                                                    ❸

    using (var context = new AppDbContext(options))                  ❹
    {
        context.Database.EnsureCreated();                            ❺
        context.Recipes.AddRange(                                    ❻
            new Recipe { RecipeId = 1, Name = "Recipe1" },           ❻
            new Recipe { RecipeId = 2, Name = "Recipe2" },           ❻
            new Recipe { RecipeId = 3, Name = "Recipe3" });          ❻
        context.SaveChanges();                                       ❼
    }
    using (var context = new AppDbContext(options))                  ❽
    {
        var service = new RecipeService(context);                    ❾
        var recipe = service.GetRecipe (id: 2);                      ❿
        Assert.NotNull(recipe);                                      ⓫
        Assert.Equal(2, recipe.Id);                                  ⓫
        Assert.Equal("Recipe2", recipe.Name);                        ⓫
    }
}
```

❶ Configures an in-memory SQLite connection using the special "in-memory" connection string

❷ Opens the connection so EF Core won't close it automatically

❸ Creates an instance of DbContextOptions<> and configures it to use the SQLite connection